

MARKDOWN WIKI (KARPATHY PATTERN), OKF

Date: 2026-09-09

Status: recap / design instrument (not a spec, not an implementation plan)

Origin: Cursor chat of 2026-09-09 — (1) [thecodacus/understory](#) read against folder 2 *data stores, databases, wiki, plain file storage (md, json, yaml)*; (2) explanation of [GoogleCloudPlatform/open-knowledge-format](#).

Reads: Understory README, core agent/OKF/lint/trace/dream/hot-memory/MCP-seed sources (main, last push 2026-08-24); Karpathy [LLM Wiki gist](#) (2026); OKF [SPEC.md v0.2](#); Google Cloud blogs [Introducing OKF](#) (12 Jun 2026) and [OKF v0.2 trust signals](#) (24 Jul 2026); folder 2 files 00–06 (especially 01, 04, 05, 06); [About Memory, RAG, and Graphs](#); Revival v3–v4 (Karpathy cluster, Memory MCP refusal); Social media literature review (Graphify / Tencent wiki); Huang *Claude Code Operating Model* ch. 2 and Mercer *Building Agentic AI Systems with Claude Code* ch. 12 as already used in folder 2 file 01 §20.2.

This document **paraphrases**. It does not paste books, the OKF spec, or chat dumps into the second brain.

Each block is asked the same four questions used on this shelf:

- 1. What was actually claimed (gist, spec, repo, or blog)?
- 2. How is it implemented in the wild?
- 3. Where does it sit on ASC / Projet Complexe / folder 1 / folder 2?
- 4. Steal, adapt, or refuse?

0. VERDICT IN ONE PAGE

Used (pattern), refused (product). The Karpathy LLM wiki, Google’s Open Knowledge Format (OKF), and Understory are three layers of the same 2026 object: *compile knowledge into a portable folder of markdown + YAML, maintain it with an agent, lint the graph*. Folder 2 already chose files-as-wiki with **pc** as the only writer. Folder 1 already named compile-at-ingest and already refused a wiki *service* as system of record. Revival v4 already listed “Memory MCP / wiki daemon (Letta, Mem0, Karpathy gist forks)” as reinventing. This pass does not reopen those refusals.

What the chat adds, that folder 2’s wiki chapter had not named:

- 1. **OKF as a public interchange spec** (v0.1 June 2026, v0.2 July 2026) — markdown lingua franca for catalog knowledge, so a stranger’s agent can parse a bundle without PC’s ontology. Treat as **export/import**, not as the Claim/Gap/Term register (D51).
- 2. **OKF v0.2 trust-in-frontmatter** — [generated](#) / [verified](#) / [status](#) / [stale_after](#) / [sources](#), plus Attested Computation. Closest published vocabulary to “once agents write the corpus, can it be trusted?” which is D04/HITL stated by Google Cloud.
- 3. **Understory as a runnable OKF v0.1 + Karpathy instance** — MCP memory tools, deterministic bundle layer, graph UI, query-path traces, seed overview, hygiene/dream loop, hot memory. Not a store to adopt.
- 4. **A job mismatch**: OKF/Understory are a **data catalog / operational wiki** (tables, APIs, playbooks, metrics). Projet Complexe is an **argumentative** second brain (Claims, typed Links, Gaps, Evidence, HITL). Do not conflate.

Karpathy gist (pattern, April 2026)
↓ formalized as
OKF v0.1 / v0.2 (format, Google Cloud, Apache-2.0)
↓ consumed by
Understory (MCP wiki daemon) Knowledge Catalog (Dataplex ingest)
Graphify, Tencent wiki, ... Obsidian / Foam as viewers
↓
PC: extract-once + HITL Claims in files + Postgres projection
export OKF (and JSON-LD) if a stranger’s agent needs a bundle
never: Understory/Letta/Mem0 as SoR

Ideal relation to the stack already decided:

Object	Role
pc / Markdown + YAML (Claims, Gaps, Terms, Sources)	Human-readable current knowledge (folder 2 D06 v6, file 04)
brain/events/*.jsonl	SoR for knowledge-state <i>history</i>
Postgres / Meilisearch / pgvector	Derived projections
OKF bundle	Optional published shape of a scope or of catalog-like extracts (tables/playbooks), not the Claim schema
Understory / other OKF agents	Negative example of agent-as-writer; steal MCP seed, query-path traces, hygiene <i>signals</i>

JARGON NOTES

Term	Alternative notations, synonyms	Definition in this note	Examples
LLM wiki / Karpathy pattern	LLM Wiki gist; compile-at-ingest; compile once	Raw sources stay immutable; an LLM incrementally writes a persistent, interlinked markdown wiki; <code>index.md</code> + <code>log.md</code> ; periodic lint. Knowledge is compiled, not re-derived on every RAG query.	Karpathy 2026 gist; Graphify; Tencent wiki asset; Understory
OKF	Open Knowledge Format	Vendor-neutral spec: a bundle = directory of UTF-8 markdown concepts with YAML frontmatter. <code>type</code> is the only always-required field. v0.2 adds optional trust/provenance/lifecycle and Attested Computation.	open-knowledge-format <code>SPEC.md</code>
Bundle	Knowledge bundle	Unit of distribution: a directory tree, a git repo, or a tarball of concepts + optional <code>index.md</code> / <code>log.md</code> .	<code>bundles/ga4/</code> ; Understory <code>BUNDLE_ROOT</code> ; <code>sample-bundle/</code>
Concept	OKF concept; page; wiki page	One <code>.md</code> file = one unit of knowledge. Path minus <code>.md</code> is the concept id. May describe a tangible asset (table, API) or an abstract idea (metric, playbook).	<code>tables/orders.md</code> ; Understory “Billing API”
Frontmatter	YAML FM	Structured block between <code>---</code> fences. Cheap to filter; body is expensive to read.	OKF <code>type</code> , <code>generated</code> , <code>verified</code> ; PC Claim fields
<code>index.md</code>	Catalog index	Reserved filename. Directory listing for progressive disclosure. Not a concept. Root may declare <code>okf_version</code> .	Karpathy catalog; Understory regenerated indexes
<code>log.md</code>	Directory update log	Reserved. Newest-first chronological history of the scope. Prose; leading bold word (<code>Update</code> , <code>Creation</code>) is convention.	Karpathy log; OKF \$9
Conformance in code	Bundle layer vs prompt	Deterministic code validates/regenerates reserved files and sandboxes paths; the LLM chooses <i>what</i> to change.	Understory <code>packages/core</code> OKF layer; PC <code>pc lint</code> + format register
Seed memory	MCP <code>instructions</code> ; instinct to look	Compact overview of the bundle injected at session start so a client model <i>thinks to query</i> memory.	Understory <code>seed.ts</code>
Query-path trace	Traversal notation	Record of search → read → write hops for one agent run, replayable on a graph. Different from folder 2 file 05’s call/HITL envelope.	Understory <code>.traces/*.json</code>
Dream / <code>memory_maintain</code>	Consolidation pass; sleep	Deterministic lint signals (orphans, broken links, duplicates, oversized pages) optionally drive an LLM repair pass. No tokens if no signals.	Understory <code>dream.ts</code>
Hot memory	Working set in front of deep memory	Last-N written <i>paths</i> (re-read fresh) + recent Q&A; cheap tool-free call; miss → full agent loop.	Understory <code>hot-memory.ts</code>
Attested Computation	OKF type; sanctioned compute	Concept that carries not only meaning but a blessed way to compute a value, plus a no-LLM attester over a run receipt. Runtime artefacts are not stored in the bundle.	OKF v0.2 \$10; <code>computations/revenue.md</code>
Trust tier	unverified / machine-confirmed / human-reviewed	Derived from <code>verified[]</code> actors, not stored. Advisory, not ACL.	OKF \$5.3
<code>stale_after</code>	Absolute freshness	Instant after which content is stale. Absolute date, not a relative TTL, so a non-LLM consumer can compare <code>now >= stale_after</code> .	OKF \$5.5

Term	Alternative notations, synonyms	Definition in this note	Examples
Understory (thecodacus)	ustory; OKF Knowledge Agent	TypeScript MCP + web UI that is an OKF v0.1 wiki daemon. Distinct from understory-io (bookings).	thecodacus/understory
Knowledge Catalog	Dataplex successor	Google Cloud product that can ingest OKF. Different repo: GoogleCloudPlatform/knowledge-catalog .	Product, not the spec
Link graph (Karpathy-style)	Generated wiki graph	Graph of <i>pages</i> compiled at ingest. Not the conceptual graph of typed Claims.	Folder 1: adapt as compiled Notes, never SoR
Conceptual graph	Claims–Links–Gaps	Meaning graph with types, status, rationale. PC’s knowledge plane.	Folder 2 file 04; Postgres recursive CTE

1. WHAT THIS CONVERSATION WAS FOR

Two questions, one afternoon:

1. Does [thecodacus/understory](#) bring something new to folder 2 (*data stores, databases, wiki, plain-file storage*)?
2. What is [GoogleCloudPlatform/open-knowledge-format](#), exactly?

They are the same cluster. Understory’s README says it “bundles follow the Open Knowledge Format (OKF) v0.1 spec” and that its graph-health design “mirrors the pattern in Karpathy’s LLM Wiki.” Folder 2 file 04 compared Obsidian, Foam, Logseq, Dendron, Org-roam, Zim, TiddlyWiki — and never named Karpathy or OKF. Folder 1 and the social-media review already had the Karpathy cluster. This note joins those two shelves.

Not this conversation: understory-io’s Rust MCP (bookings/events API). Same word, different product.

2. THE KARPATY LLM WIKI PATTERN

2.1 WHAT WAS ACTUALLY CLAIMED

Karpathy’s 2026 gist is an *idea file* to paste into an agent, not a product. The claim against classic RAG (NotebookLM, ChatGPT file upload, most vector RAG):

The LLM is rediscovering knowledge from scratch on every question. There is no accumulation.

Instead: incrementally **build and maintain a persistent wiki** of markdown files that sits between the human and the raw sources. When a source arrives, the LLM reads it, extracts, integrates (update entity pages, note contradictions, strengthen synthesis). Cross-references and contradictions are already there at query time. “The wiki is a persistent, compounding artifact.”

Three layers:

Layer	Who writes	Mutability
Raw sources	Human curates	Immutable; LLM reads only
Wiki	LLM owns	Created, updated, cross-linked
Schema	Human + LLM co-evolve	CLAUDE.md / AGENTS.md : conventions and workflows

Operations: **Ingest** (one source may touch 10–15 wiki pages), **Query** (search pages, cite; good answers may be filed back), **Lint** (contradictions, stale claims, orphans, missing pages, missing cross-refs). Special files: [index.md](#) (content catalog — titles, one-liners, categories; read first at query time) and [log.md](#) (chronological, parseable prefixes). Search: index file until ~hundreds of pages; then something like qmd (BM25 + vectors). Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase. Git is free history. Why it works, as claimed: humans abandon wikis because bookkeeping grows faster than value; LLMs do not get bored, forget a backlink, or refuse to touch fifteen files. The human’s job is sources, questions, meaning. Related to Bush’s Memex: private, curated, associative trails; the unsolved part was *who maintains*.

The gist is intentionally abstract: directory layout, schema, tooling are the reader’s.

2.2 HOW IT IS IMPLEMENTED IN THE WILD

Named in this household’s earlier notes, not re-argued:

- **Graphify** — folk productization ~48 h after the gist: folder → Obsidian vault with backlinks; claimed token compression vs reading raw files. Revival / social-media review: compilation move, not the SoR.
- **Tencent markdown pyramid** — compile at ingest; filesystem as truth; persona auto-rollup refused.
- **Hermes MEMORY.md** — bounded always-on file; packing lesson, not semantic memory.
- **Understory** — this chat’s worked example (§4).
- **OKF** — Google’s attempt to make the *file conventions* interoperable (§3).

2.3 WHERE IT SITS ON PC

Already decided, restated so this file stands alone:

- **About Memory, RAG, and Graphs**: Karpathy-style link graph = compiled Notes, **never SoR**. Adapt as: extract once to canonical text; optional compiled notes; never replace files with a wiki *service*.
- Revival v4 wheel-reinventing test: **Memory MCP / wiki daemon** (Letta, Mem0, Karpathy gist forks) = already refused.
- Folder 1 D18: extract-once; indexes fan out. That is compile-at-ingest for *sources*. Claims are not auto-compiled: D04, HITL.
- Folder 2 D58: stop promising emergence; schedule confrontation (IBIS lints). Karpathy’s lint *includes* contradictions, but also “suggest new questions” and growth-by-linking — the marketing half of PKM C4 that file 04 dropped.
- Folder 2 D35 / file 01 §20.2 (Mercer): no opaque agent auto-memory as SoR. A *transparent* markdown wiki written by an agent without HITL is still the wrong **writer**, even if the bytes are inspectable.

The pattern’s valid half for PC: **compile once** (extract-once, optional generated `views/`, nightly consolidation that *proposes*). The invalid half: **the LLM owns the wiki**.

2.4 STEAL, ADAPT, OR REFUSE

Move	Verdict
Compile at ingest vs RAG-every-query	Steal — already D18
<code>index.md</code> + <code>log.md</code> as navigation + chronology	Adapt — PC’s <code>views/</code> are generated; decision log is JSONL not prose <code>log.md</code> ; do not make <code>index.md</code> the query engine once Meilisearch exists
Lint orphans / broken links / stale claims	Adapt — IBIS lints are the knowledge form; wiki-hygiene lints (dangling <code>[[slug]]</code> , oversized note) can sit beside them as views
LLM writes the wiki; human only sources and questions	Refuse as SoR writer. Worker proposes; <code>pc</code> writes; human accepts (D04, D35, file 04 “ <code>pc</code> is the only writer”)
File answers back into the wiki	Adapt as <i>proposed</i> Claims / notes, never auto-accepted
Obsidian as IDE	Already — file 04: viewer to target, not the store; readable-by test; no Dataview/Canvas/block-refs

3. OPEN KNOWLEDGE FORMAT (OKF)

3.1 WHAT WAS ACTUALLY CLAIMED

OKF is a format, not a platform. Announced 12 June 2026 by Google Cloud Data Cloud (Sam McVeety, Amir Hormati). Problem: internal knowledge (schemas, metric meanings, runbooks, join paths, deprecations) is fragmented across catalogs, wikis, comments, and heads; every vendor reinvents a catalog API. Agents reassemble the same context from scratch.

The missing piece is not another knowledge service. It is a representation that:

- anyone can produce without an SDK;
- anyone can consume without an integration;
- survives moving between systems;
- lives in version control beside the code it describes;
- is readable by humans and parseable by agents — same file, no translation.

If you can `cat` a file, you can read OKF; if you can `git clone` a repo, you can ship it. No schema registry, no central authority, no required tooling.

Three design principles (v0.1 blog):

1. **Minimally opinionated.** Exactly one required field: `type`. Types, extra keys, body sections are the producer’s.
2. **Producer/consumer independence.** A human-authored bundle, a catalog export, and an LLM-synthesized bundle are the same contract.

3. **Format, not platform.** Not tied to a cloud, model, or agent framework. Never requires a proprietary account to read.

Google explicitly cites Karpathy. OKF “formalizes the LLM-wiki pattern into a portable, interoperable format.”

Non-goals (spec §1): a fixed taxonomy of types; storage/serving/query infrastructure; replacing Avro / Protobuf / OpenAPI (OKF *references* them via *resource*); a packaging/invoke standard for executor/attester code.

3.2 THE TWO GOOGLE REPOS (DO NOT MIX)

Repo	What it is	Stars (as of 2026-09-09)	Role
GoogleCloudPlatform/open-knowledge-format	Spec (SPEC.md v0.2) + Python reference-agent (producer + viz.html consumer) + sample bundles	~365	The format
GoogleCloudPlatform/knowledge-catalog	Google Cloud Knowledge Catalog (formerly Dataplex) tools and samples	~9k	A product that can ingest OKF

OKF v0.1 first lived as `okf/SPEC.md` under knowledge-catalog; the dedicated repo (created 2026-08-11) is the spec home. Understory still points at the knowledge-catalog path for v0.1.

License: Apache-2.0.

3.3 BUNDLE STRUCTURE (SPEC §§2–4, 8–9)

A **bundle** is a directory tree of markdown files. Identity of a concept = path within the bundle with `.md` stripped.

Reserved filenames (must not be used as concepts), at any level:

Filename	Purpose
<code>index.md</code>	Directory listing for progressive disclosure (§8)
<code>log.md</code>	Update history of that scope (§9)

All other `.md` files are concepts.

A concept is UTF-8 markdown with:

- 1. YAML frontmatter (`--- ... ---`);
- 2. a markdown body.

Required: `type` — short string for routing/filtering/presentation. Examples: `BigQuery Table`, `BigQuery Dataset`, `API Endpoint`, `Metric`, `Playbook`, `Reference`, `Attested Computation`. Types are **not** registered centrally. Consumers **must** tolerate unknown types (treat as generic). A file with only `type` is fully conformant.

Recommended: `title` (else derive from filename), `description` (one sentence; used by indexes and snippets), `resource` (canonical URI of the underlying asset; absent for abstract ideas), `tags` (YAML list).

Extensions: producers may add any keys. Consumers **should** preserve unknown keys on round-trip and **must not** reject unrecognized fields.

Body: standard markdown. Prefer structure (headings, lists, tables, fenced code). No required sections. Conventional headings when they apply: `# Schema`, `# Examples`, `# Computation` (v0.2). Per-claim attribution uses footnotes keyed to `sources[].id`, not a body `# Citations` list (retired in v0.2).

Identity and links (§6):

- Recommended link form: bundle-absolute `[text](/tables/customers.md)` (stable if files move within a subdirectory).
- Also allowed: relative `./other.md`.
- A link asserts a *relationship*; the *kind* (joins-with, depends-on, parent) is in surrounding prose, not typed on the edge. Graph viewers typically treat all links as untyped directed edges.
- Consumers **must tolerate broken links** (may mean not-yet-written knowledge).
- Path-valued fields (`resource`, `sources[].resource`, executor/attester paths) accept absolute URL, bundle-absolute `/...`, or relative path. A `sources[].resource` may instead be a *scope descriptor* (“all queries in project X”), which is not a path.
- `references/` is a **naming convention** (not required) for mirrored external material, run instructions, attester code.

`index.md`: optional, any directory. No frontmatter except the bundle-root file may carry `okf_version`. Sections with bullet lists `* [Title](url) - description`. Producers may generate; consumers may synthesize if missing. This is Karpathy’s “read the index first” and Hermes-style progressive disclosure.

`log.md`: optional, any level. Newest-first, `## YYYY-MM-DD` headings. Entries are prose; `**Update**` / `**Creation**` / `**Deprecation**` are convention.

Distribution: git (recommended), tarball/zip, or a subdirectory of a larger repo.

3.4 V0.2 — PROVENANCE, TRUST, LIFECYCLE (SPEC §5)

v0.1 was enough for “a folder of typed markdown.” v0.2’s motivation (blog 24 Jul 2026): the valuable bundles will be **written continuously by agents** and consumed by *other* agents. A human-authored wiki page carries an implicit guarantee (a person wrote it; you can hold them accountable). That guarantee is gone at ten thousand generated concepts. The consumer must answer five questions **from frontmatter**, without spending tokens on the body:

- 1. What was this created from? (**provenance**)
- 2. How much should I trust it? (**trust**)
- 3. Is it still true? (**freshness**)
- 4. Is it the current version? (**lifecycle**)
- 5. Was this number produced the way we said it must be? (**attestation**, §3.5)

All of these families are **optional**. Absence carries meaning (unverified ≠ verified) but **must not** cause rejection (§11). `type` remains the only always-required field. Custom keys still preserved. Every timestamp in OKF is ISO 8601 with explicit UTC offset.

Provenance: sources

List of materials the concept derives from (external URL, bundle-relative path, or a scope descriptor). Each entry:

- `resource` — required within an entry;
- `id` — optional stable key for footnotes; **should** be present if the body cites;
- `title`;
- credibility **signals** (not a score): `author`, `usage_count`, `last_modified`;
- sibling `usage_window: { from, to }` framing counts.

OKF **records signals, not a credibility score**. A score is subjective, unportable, and stale. Consumers infer. `usage_count` is liveness/trend, not a precise ranking across kinds (a scheduled query’s executions ≠ a human’s dashboard views). Lineage of OKF-to-OKF derivation is **links**, not a `derived_from` field. Deeper external lineage is out of scope for v0.2. Footnotes: `[^ga4-schema]` joins to `sources[].id`. Labels are keyed, not positional — agents reorder lists; `sources[0]` would silently misattribute.

Trust: generated and verified

Kept distinct: who *wrote* need not be who *confirmed*.

```
generated: { by: reference_agent/gemini-2.5-pro, at: 2026-06-20T22:53:05Z }
verified:
- { by: human:ahormati, at: 2026-06-25T09:00:00Z }
  - { by: process:finance-nightly, at: 2026-06-26T02:00:00Z }
```

A bare mapping `verified: { by, at }` must be treated as a one-element list. `verified` is independent of `generated.at`: content can change without re-confirmation; facts can be re-confirmed without regeneration.

Actor convention (§7)

Form	Meaning
<code><producer>/<version></code>	Agent or tool (<code>reference_agent/gemini-2.5-pro</code>)
<code>human:<id></code>	Person (<code>human:ahormati</code>)
<code>process:<id></code>	Automated process (<code>process:finance-nightly</code>)

Trust classification keys off the `human:` prefix; producers **must** use it for hand-authored or human-confirmed content.

Trust tiers (§5.3) — derived, lowest to highest

<code>verified</code>	Tier
absent	unverified
only non- <code>human:</code> actors	machine-confirmed
any <code>human:<id></code>	human-reviewed

Tiers are advisory, not access control. A concept with no trust frontmatter remains consumable.

Lifecycle: `status` and `stale_after`

`status`: `draft` (not yet reviewed) | `stable` (default if absent) | `deprecated` (kept for links/history).
`stale_after`: optional **absolute** instant. Stale when `now >= stale_after`. Absolute, not relative TTL, so a deterministic consumer needs no “time of read.”

3.5 ATTESTED COMPUTATION (SPEC §10)

The distinctive v0.2 object. Provenance answers “where did this claim come from.” Attestation answers “was **this number** produced the sanctioned way, or did the agent improvise SQL?”
OKF records the computation and how to check it. **It does not execute.**
A sanctioned computation is its own concept (`type: Attested Computation`), not a field on a Metric, for three reasons: `runtime` defines what `parameters` mean; one computation can back many consumers; trust/staleness/attester are per computation (revenue can be fresh while profit is stale).
Contract fields (in addition to §5 families):

Field	Role
<code>runtime</code>	Required. How to run: <code>bigquery</code> , <code>postgres</code> , <code>dbt</code> , <code>python</code> , <code>Looker</code> , ...
<code>parameters</code>	Typed named holes the agent may fill <code>{ name, type, required }</code> . Agent must not author or edit the computation.
<code>computation</code>	Optional path to a file; else the body <code># Computation</code> fence is the computation
<code>executor.resource</code>	Run instructions or code
<code>executor.receipt</code>	Fields a run must return (e.g. <code>job_id</code> , <code>executed_sql</code> , <code>result</code>) — evidence the attester inspects
<code>attester.resource</code>	Deterministic, no-LLM code: receipt → verdict. Meant to run consumer-side

What sits behind a `resource` (Skill, script, container) is a packaging choice. OKF fixes the interface.
Informative consumer loop: discover by type → load contract + computation → parameterize → executor returns receipt → attester checks (computation that ran equals bound sanctioned computation; displayed value matches receipt, not the model’s prose) → gate (refuse display on fail; warn/refuse if stale).
Verification vs attestation:

	<code>verified</code>	Attestation
Confirms	The <i>definition</i> still matches policy	A single <i>run</i> produced the value the sanctioned way
When	Doc-level, slow	Per-call, runtime
Stored in bundle?	Yes	No

A stale definition can still attest; a freshly verified definition still needs attestation on every run.
Deferred (spec §12): full runtime protocol (receipt/verdict wire formats), attester ABI/sandboxing, attestation caching, semantic-layer (Looker/dbt) equality instead of SQL equality.

3.6 CONFORMANCE AND VERSIONING (SPEC §§11–13)

A bundle is conformant with v0.2 if:

- every non-reserved `.md` has parseable YAML frontmatter;
- every frontmatter has non-empty `type`;
- reserved files, if present, follow §8–§9.

Consumers **must not** reject for: missing optional fields, unknown `type` values, unknown extra keys, broken cross-links, missing `index.md`.
When trust/lifecycle/computation families *are* present, producers should follow §5–§10; consumers must treat bare `verified` as a list, must not reject missing optional families, should derive tiers/staleness only from specified fields, and should surface (not silently drop) a failing attestation.
Versioning: `major.minor`. Minor = backward-compatible additions. Major = breaking (rename required fields, change reserved names).
Bundles may declare `okf_version: "0.2"` on root `index.md`. Unknown version → best-effort consume, do not refuse.
v0.1 → v0.2 breaking (with fallbacks):

v0.1	v0.2
timestamp	generated.at (fallback to timestamp if generated absent)
body # Citations	sources frontmatter (fallback: still parse the old list)

Additive: `sources` + credibility signals, `generated/verified`, `status/stale_after`, type Attested Computation and its keys, heading `# Computation`, actor convention.

A v0.1 bundle is consumable by a v0.2 consumer under those fallbacks. A bundle that adopts none of the new keys is still valid v0.1.

3.7 HOW IT IS IMPLEMENTED IN THE WILD

Reference agent (this repo, Python ≥3.11, Google ADK, BigQuery client, Gemini): proof-of-concept **producer**. Two passes:

- 1. **BQ pass** — one OKF doc per concept the source advertises, from BigQuery metadata alone.
- 2. **Web pass** — LLM as crawler: seed URLs, `fetch_url`, follow only if they look like authoritative docs for *existing* concepts; then enrich, mint `references/`, or skip. Hard `--web-max-pages` and same-domain allowlist inside the tool. `--no-web` skips.

Recipes live in `samples/<name>/`; produced trees in `bundles/<name>/` (GA4 e-commerce, Stack Overflow, Bitcoin; `acme_retail` for v0.2 trust fields).

Visualizer: `python -m reference_agent visualize --bundle ...` writes a **self-contained** `viz.html` (Cytoscape.js + marked from CDN; bundle inlined as JSON; no backend; no data leaves the page). Force-directed graph colored by type, detail panel, backlinks, search, type filter, layouts.

Independent consumers: Understory (v0.1, \$4); Rust `okf-core`; anything that already reads markdown + YAML (Obsidian, MkDocs, Hugo, an LLM). Community extension proposals mentioned in the v0.2 blog (not in the spec yet): typed relationship edges, agent-routing hint fields, optional erasure conformance profile, `.okfignore`.

3.8 WHERE IT SITS ON PC

Folder 2 file 04 already asked: what does the brain look like to Obsidian, a graph query, Zotero, an SSG, **and a stranger’s agent**? The answers written there are: CommonMark + YAML + typed wikilinks in front matter; RDF/JSON-LD/PROV-O/Web Annotation **export-only**; no triple store.

OKF is the **markdown-shaped** answer to the stranger’s-agent question. It is not a replacement for:

- the format register (D51) — OKF has no `schema`: integer, no closed key set, no `kind` of Claim/Gap/Term;
- typed Links with `rationale` (D02) — OKF edges are untyped prose;
- HITL state machine (D04, D54, file 02 `transition()`) — OKF `verified/status` are *signals*, not a role-constrained state machine;
- bi-temporal valid/transaction time — `stale_after` is a single cut; `generated.at` is last meaningful change, not a sys-period;
- Zotero as bibliographic SoR (D57).

Job mismatch, stated once: **OKF is catalog knowledge** (what is this table, how is this metric computed, what is the on-call playbook). **PC is argumentative knowledge** (this statement holds, with Evidence, against that statement, answering this Gap). A published *catalog* extract (CodeGraph-adjacent APIs, client playbooks, published Terms) can be OKF. A Claim file should not become `type: Decision` with free-form body as the SoR.

Four layers mapping (ontology / semantics / dynamics / execution): OKF standardizes a **document ontology** (concept = file, id = path) and a little **semantics** in frontmatter (`type`, `trust`). It does not model dynamics (supersession chains, Gap lifecycle) or execution (ASC hooks).

Attested Computation is the one execution-adjacent object, and even there OKF only *describes* the interface.

Folder 2 file 01 YAML hygiene (strict 1.2 profile, no Norway problem, `schema`: integer) is **stricter** than OKF’s “any extra keys, consumers must not reject.” Exporting OKF from PC is a serializer problem (map PC fields onto OKF keys, put the rest in a `pc`: extension namespace that consumers preserve). Importing OKF as Claims would be a lossy promotion through HITL — same as Hypothesis/Zotero annotation import (file 04 §9.3).

3.9 STEAL, ADAPT, OR REFUSE

Move	Verdict
Files + YAML FM + git as the interchange surface	Already D06 / file 04
<code>type</code> as the only required field; free-form types	Refuse for PC objects. Closed kinds + <code>schema</code> : (D51). OKF <code>type</code> is fine <i>inside an exported bundle</i>
Preserve unknown keys; do not reject	Steal for <i>import</i> of foreign OKF; PC’s own writer stays strict

Move	Verdict
Trust in frontmatter so a packer can filter before reading the body	Steal the split — FM = cheap filter, body = expensive read. PC already puts typed fields in FM for this reason. Vocabulary to consider: <code>generated/verified</code> vs collapsing into <code>extracted_by</code> + <code>status</code> ; <code>stale_after</code> as absolute date for lints
Signals not scores	Steal — same instinct as D04 (consensus is not truth) and file 04’s refusal of Kialo impact votes
<code>human:</code> / <code>process:</code> / <code>agent/version</code> actors	Adapt — maps onto <code>who</code> , provider, <code>hitl</code> vs extractor
Attested Computation	Adapt into the harness , not the wiki. Sanctioned SQL/recipe + deterministic check is folder 1 (<code>able</code> , recipes, <code>pre_llm</code>). Do not store receipts in <code>pc/</code>
Untyped markdown links; broken links tolerated	Refuse for the SoR. File 04: typed links; readable-by test fails on dangling. Broken links as “not yet written” is a catalog habit
Directory taxonomy (<code>tables/</code> , <code>apis/</code>) as identity	Refuse for Claims (flat folders keyed by id, C2). Fine for an OKF <i>export</i> of catalog-like material
<code>index.md</code> progressive disclosure	Already packing/LOD; generated <code>views/</code>
<code>log.md</code> as prose history	Refuse as knowledge-state SoR (decision log is JSONL, D54). A generated <code>log.md</code> in a published OKF bundle is fine
Knowledge Catalog / Dataplex as store	Refuse — another service. D06
OKF as published-bundle format beside JSON-LD/PROV-O	Adapt — file 04 addendum candidate: recommended source, export-only, fixture test
Community “typed relationship edges”	PC already has them; if OKF grows typed edges, map <code>supports/contradicts</code> /CiTO on export
Erasure profile / <code>.okfignore</code>	Watch; maps to forgetting/tombstones (files 01, 05) if it lands

4. UNDERSTORY (THECODACUS)

4.1 WHAT WAS ACTUALLY CLAIMED

Tagline: “Memory that grows.” A **self-wiring, plain-markdown memory** for AI agents: MCP + local models + a living graph. Every fact an agent learns is filed as a markdown concept, cross-linked, kept healthy by the agent, searchable, diffable, entirely local. Runs on llama.cpp. Design rule, quoted because it is the architectural sentence:

Conformance is enforced in code, not prompts. The deterministic bundle layer validates frontmatter (`type` required), regenerates `index.md` files, appends `log.md` entries (newest-first, spec §7), and sandboxes all paths to the bundle root. The LLM decides *what* to change; the code guarantees the result is a conformant bundle.

Three ways in, one internal agent: MCP (`memory_query` / `memory_add` / `memory_update` / `memory_status` / `memory_maintain`), web UI (tree, concept viewer, log, conformance badge, force-directed graph, chat), CLI smoke (`agent:query` / `agent:mutate`). Each MCP call drives an **internal** LLM with the OKF spec in the system prompt. Apache-2.0. Created 2026-07-09; last push 2026-08-24. ~315 stars, TypeScript pnpm monorepo (`packages/core`, `server`, `web`). Not a fork of understory-io.

4.2 HOW IT IS IMPLEMENTED

Stack

Package	Role
<code>packages/core</code>	OKF bundle layer (zero LLM) + agent (Vercel AI SDK tool loop: search/read/list/write/patch/delete) + provider registry
<code>packages/server</code>	Express: MCP streamable HTTP <code>/mcp</code> , stdio bin, REST <code>/api/*</code> , streaming chat, serves the web build
<code>packages/web</code>	Vite + React + Tailwind: browser + graph + chat

Providers: any OpenAI- or Anthropic-compatible API (`LLM_API_BASE_URL`, `LLM_API_KEY`, `LLM_API_FORMAT`, `LLM_MODEL`). Fallback pair. llama.cpp via `--jinja` tool calling; llama-swap: prefer currently loaded model so a query does not trigger a swap. Docker image `ghcr.io/thecodacus/understory`; `BUNDLE_ROOT` is the volume. Optional `AUTH_TOKEN` (Bearer) for `/mcp` and `/api`; stdio needs none. Optional `GIT_AUTO_COMMIT=true` commits every mutation.

Tests: ~18 core tests (spec §§5–7, 9, sandbox, search, concurrency). MCP smoke needs a key.

Sample bundle (OKF v0.1 catalog, not a Claim brain)

```
sample-bundle/
index.md      # okf_version: "0.1"; lists segments
log.md        # dated Creation / Update
apis/         # e.g. Billing API
playbooks/    # e.g. Billing On-call
tables/       # e.g. Customers
```

Subject-area directories. Concepts cross-link with bundle-absolute markdown links.

System prompt (Knowledge Keeper)

Rules worth recording because they *are* the Karpathy schema operationalized:

1. **Search first** before adding.
2. **Enrich over create** — a fact that is an attribute of an existing entity is patched *into* it.
3. **Link both ways** — new concept plus backlinks from related concepts; unlinked = invisible.
4. **Reuse types** already in the bundle.
5. **Place deliberately** — directories by subject; kebab-case filenames.
6. **Write for the next reader** — self-contained bodies; one-line `description`.
7. **Prefer patch over rewrite**.
8. **Deprecate, don't delete** (unless wrong/harmful or user asks).
9. **Log summaries** — one past-tense sentence with bundle-relative links.
10. **Cite when answering** — never invent; say if the bundle has no coverage.

Query mode: keyword search is not semantic; empty search proves nothing; retry synonyms; then read plausibly related concepts from the tree; only then “not found.”

Mutate mode: **check for contradiction**. Do not leave two claims standing; do not silently drop the old one. Update and make supersession explicit. “The old statement must no longer appear anywhere in the concept.” If it sits in prose, rewrite the **whole body**. This is in-place erasure, not PC supersession.

Graph, lint, search

`graph.ts`: one pass; reserved `index.md/log.md` are **not** link sources (their catalogs would drown real edges). Regex on `](/path.md)`. Broken links listed; inbound degree per concept.

`lint.ts`: **deterministic, no LLM** — orphans (inbound 0) + broken links. `healthy` iff both empty. Karpathy's anti-drift lint.

Search: naive scan (`search.ts`); README defers hybrid FTS5+embeddings “until the low thousands of concepts.” Same staging as Karpathy's index-first and folder 2 file 03.

MCP seed (`seed.ts`)

A client that only sees four tool names never checks memory. At session start Understory injects a compact overview (types, per-segment descriptions not just filenames, recent log, truncated (~3000 chars), via:

1. MCP initialize `instructions`;
2. `memory_query` **tool description** (universal fallback);
3. after writes in long-lived stdio, `tools/list_changed` so the session sees its own writes. Hand edits / other clients: next session.

This is packing, not storage: “semantic hooks beat filenames for igniting the instinct to look.”

Query-path traces (`trace.ts`)

Every query/mutation/chat records steps (tool, summary, paths, write flag), duration, truncated input/answer, model chain, optional token usage. Compact notation, e.g. `search "rate limit" (2) → read billing-api.md → ✓`. Persisted as JSON under `<bundle>/.traces/` — a **dot-directory OKF walkers ignore**, so traces never enter conformance or the graph. Pruned to **50** files. Telemetry must not fail the run. The web graph can replay a run as numbered hops (visited ringed, search hits dotted, rest faded).

This is **not** folder 2 file 05's envelope (ten fixed fields, catalogue, PII class, retention, DuckDB). It is a **wiki traversal** derived view.

Dream / maintain (`dream.ts`)

Autonomous consolidation “what a brain does during sleep.” **Deterministic signals first; no signals → no run, no tokens.**

Signal	Detection	Intended repair
Orphans	inbound = 0	Wire into related concepts, or leave if unrelated
Broken links	target missing	Fix path or remove
Likely duplicates	Jaccard on title+description tokens, threshold 0.65, cap 5, O(n²)	Merge into better-placed, retarget links, delete duplicate if true duplicate ; else cross-link
Oversized	body ≥ 6000 chars or ≥ 6 # sections; max 3 per dream	Split hub-and-spoke : extract subtopics to new concepts, rewrite original as hub, never delete or rename original path (inbound links survive)
Optional insights	DREAM_INSIGHTS not false and log.md ≥ 5 entries	If several concepts share a theme with no overview, create one; do not force an insight

memory_maintain = lint report drives the internal agent. Write-time linking is the other half: enrich vs create; contradictions superseded in place.

Hot memory (hot-memory.ts)

Working set in front of the deep loop. Module-level (survives stateless HTTP MCP instances):

- last 10 written paths, timestamps; re-read fresh at lookup (never a stale snapshot of body);
- last 10 Q&A pairs;
- TTL default 1h (HOT_MEMORY_TTL);
- any write clears Q&A (write may contradict answers);
- one tool-free LLM call over excerpts (max 1500 chars/concept); must answer UNKNOWN if not confident → fall through to deep agent;
- disable with HOT_MEMORY=false.

Inspectable (paths into files). Not Mercer’s opaque cache. CoALA working memory in front of the wiki.

Graph UI

Force-directed: color by type, size by degree, orphans ringed red, click to open. Path replay over that graph. File 04 already called Obsidian’s graph a toy; this is the same toy with overlay. The replay is the interesting bit, not the physics layout.

4.3 WHERE IT SITS ON FOLDER 2 (THE ORIGINAL QUESTION)

Does it bring something interesting to the data-stores research? Yes as a worked example and as an OKF consumer; no as a store.

Folder 2 decision	Understory	Verdict
D06 files SoR; engines rebuildable	Files are the store. No Postgres, no derived_state, no rebuild drill	Confirms the file side. Adds nothing on derived stores, hashing, HWM, drill
D54 accepted Claims immutable; change = supersession	Contradictions rewritten in place; old sentence must vanish from the body	Conflict. Their “supersede” is an edit, not a new object with superseded_by
D35 / pc only writer; no opaque auto-memory	Nested agent writes on memory_add. Files are inspectable; writer is unconstrained	Transparent bytes, wrong writer. Revival v4 already refused this class
D58 schedule confrontation; no emergence	Hygiene lint is useful; dream’s optional “overview if a theme emerges” is PKM C4 in code	Steal signals; refuse autonomous insight-write
D51 format register, schema:	OKF: only type; free-form; unknown keys kept	Too loose for PC objects; interesting as export
File 04 C2 typed links + rationale; flat id folders	Untyped body links; directories are subject areas	Catalog job, not Claim job
Readable-by test fails on dangling	Broken links tolerated at write, repaired later by maintain	Opposite of fail-closed lint
File 01 git cadence	GIT_AUTOCOMMIT per mutation	File 01 already refused per-keystroke auto-commit
File 03 staged search	Naive scan until thousands	Same staging; not news

Folder 2 decision	Understory	Verdict
File 05 traces	<code>.traces</code> of hops, prune 50, ignored by walkers	Different kind of event: query-path. Complementary derived view
Mohan / single-writer (01 §20.1)	One agent, one bundle; no ULID log, no <code>transition()</code>	Two MCP clients → Mohan’s race. Confirms the addendum
File 04 IBIS lints	Orphan <i>pages</i> and broken links, not orphan <i>issues</i> / unargued claims / un confronted pairs	Hygiene ≠ deliberation. Keep IBIS; add hygiene views if wanted
Parked ArangoDB / refuse Neo4j	No graph database	Confirms “files + optional SQL projection.” Does not move T1–T4

Internal LLM **on every** `memory_query`: the store is an agent. PC’s cascade (Meilisearch / FTS / accepted walk) is cheaper and lag-measurable. Karpathy’s compile-once is already extract-once + HITL, not a nested model on each read.

4.4 STEAL, ADAPT, OR REFUSE

Move	Verdict
Understory as the second brain / Memory MCP	Refuse — Revival v4; D35; D04
Conformance in code, not prompts	Steal the split — already the intent of <code>pc lint</code> + format register + sandbox. Understory is a small existence proof
MCP seed + tool-description refresh	Steal — folder 1 packing / MCP. One-line pointer from file 04 “stranger’s agent”
Query-path traces + graph replay	Adapt as a derived view over file 05 (or a <code>kind</code> in the catalogue), not a second ledger. Do not prune to 50 as the retention story
Deterministic lint → optional LLM	Steal the factoring (no tokens if healthy). Repair stays <code>proposed</code>
Jaccard duplicates + oversized hub-and-spoke (preserve path)	Adapt as <code>views/duplicates.md</code> / oversized lint, like <code>views/orphans.md</code> . No dream writer
Enrich-over-create; bidirectional linking at write	Adapt as <i>compiler-at-ingest agent behaviour</i> for notes/playbooks , not as Claim identity. Write-time linking without typed <code>rationale</code> is still PKM C2 uncorrected
Hot memory (fresh path reads, wipe Q&A on write)	Adapt in folder 1 packing — inspectable working set. Not a store
In-place contradiction erasure	Refuse — D54
Auto-commit per mutation	Refuse — file 01 §8.5
Force-directed graph as truth	Refuse — toy; typed backlinks + IBIS badges are the UI
Nested LLM as query engine	Refuse as default. T0/ <code>rg</code> /Meilisearch first (D17)
Local llama.cpp + OpenAI-compatible fallback	Neutral / already the provider story. Not a data-layer decision

5. CROSSWALK: CONVERSATION OBJECTS → PC OBJECTS

Their object	Closest PC object	Do not collapse
OKF concept	A <i>page</i> (note, playbook, exported Term, source note)	Not a Claim unless promoted through HITL
OKF <code>type</code>	<code>kind</code> / object class	Theirs is open; ours is closed
OKF <code>resource</code>	<code>doc_id</code> / attachment path / Wikidata QID	Catalog URI vs bibliographic SoR (Zotero)
<code>sources[]</code> + footnotes	Evidence + PROV sidecar	Theirs is page-level; Evidence is span-anchored

Their object	Closest PC object	Do not collapse
generated.by/at	extracted_by, provider, accepted_at	Writer ≠ confirmer is the split we already want
verified[] / trust tier	HITL accept; status	Tiers are advisory; PC status is a state machine with roles (D55)
status: draft/stable/deprecated	proposed / accepted / deprecated	Three words, different constraints (pc_worker can only propose)
stale_after	valid_during end / dormancy / retention	Absolute date for lints is stealable; not a substitute for bi-temporal
Attested Computation	Recipe / able + deterministic post-check	Harness, not wiki
index.md	Generated views/ + packing LOD	Not the query engine
log.md	Decision log JSONL + git	Prose log is a published render
Untyped markdown link	Note-body](slug.md) (file 06 §2.4: never creates a Link object)	Claim→Claim Links are typed, inlined in FM, logged as objects
Orphan page	IBIS orphan issue is different	Both can exist as views
Query-path .traces	File 05 envelope + window dump	Complementary kind
Hot memory	Packed working set / always-on block	Bounded; not MEMORY.md as semantic archive
MCP memory_*	pc CLI + optional MCP adapter	Transport only (About Memory jargon: MCP does not store knowledge)

6. FOUR QUESTIONS, ANSWERED ONCE FOR THE CLUSTER

What was claimed? Compile knowledge into a markdown wiki; make the wiki’s conventions a portable spec (OKF); optionally run an agent that files, links, lints, and serves it over MCP (Understory).

How is it implemented? Gist (schema in AGENTS.md) → OKF files anyone can cat → Google reference agent (BQ + web enrich + static viz) and third parties (Understory, okf-core, Knowledge Catalog ingest).

Where on the stack? Pattern: already in folder 1 as compile-at-ingest. Format: folder 2 file 04 **export** slot, next to JSON-LD. Daemon: refused as SoR; MCP is transport. Hygiene/seed/path-traces: harness + views.

Steal, adapt, refuse? Steal the FM-vs-body filter split, conformance-in-code, MCP seed, query-path view, hygiene signals, stale_after as absolute date, signals-not-scores. Adapt OKF as a published bundle and Attested Computation as a harness interface. Refuse agent-owned wiki, in-place erasure, free-form types as Claim schema, auto-commit, nested-LLM-as-index, Memory MCP as identity.

7. WHAT FOLDER 2 / FOLDER 1 WOULD CHANGE IF THIS WERE AN ADDENDUM

This file is the research note. It does **not** edit the Nextcloud folder-2 review. If a later pass writes addenda, the chat’s placement was: **Folder 2 file 00 §3.2 (recommended, not on the shelf)**

Item	Why
Karpathy, “LLM Wiki” gist (2026)	The pattern file 04’s PKM chapter did not name; compile-at-ingest vs RAG; index.md/log.md/lint
OKF SPEC v0.1 and v0.2 (GoogleCloudPlatform/open-knowledge-format)	Public markdown interchange; v0.2 trust/attestation vocabulary
thecodacus/understory	Runnable OKF v0.1 + Karpathy instance; cite as implementation, not as design

Folder 2 file 04 — addendum: OKF as export-only sibling of PROV-O/Web Annotation; Understory as negative example of agent-as-writer and positive example of seed + path replay; hygiene lints (duplicates, oversized hub-and-spoke) as extra views/ without adopting dream-write; D58 confirmed (dream “insights” = emergence).

Folder 2 file 01 — OKF compared to the format register, not adopted; “conformance in code”; Mohan addendum confirmed by multi-client MCP writes.

Folder 2 file 05 — query-path as a possible event kind or derived overlay; `.traces` ignored by wiki walkers is the right *isolation*, 50-file prune is the wrong *retention*.

Folder 1 — MCP seed / `tools/list_changed`; hot memory as inspectable working set; Attested Computation → recipes; packing: filter on FM trust/staleness before packing bodies.

Do not reopen: Memory MCP refusal; D06; D54; D55 worker-propose; Arango parked door; Zotero as bibliographic SoR.

8. OPEN QUESTIONS (HANDED, NOT DECIDED HERE)

- Export mapping.** If `pc publish` or `pc export okf` exists, which PC objects become OKF concepts (`claim as type: Claim` with `pc:` keys vs only catalog-like objects: Terms, playbooks, CodeGraph symbols, client runbooks)? Who owns `verified` — the HITL event?
- v0.2 vs Understory.** Understory is v0.1. Designing against Understory’s code would freeze the weaker trust model. Prefer the spec.
- Typed edges in OKF.** If the spec later adds them, do we export `supports/contradicts` as typed edges or keep CiTO in JSON-LD only?
- Attested Computation vs able.** Is a recipe already the executor+attester pair? If yes, OKF is a *serialization* of a subset of recipes for interchange with other agents.
- Seed size vs D17.** Injecting a 3k-char overview every session is a packing cost. When does it beat “the tool exists, the model should call it”? Measure; do not copy blindly.
- Query-path PII.** Traversal traces name concept paths and truncated user input. Catalogue them (D56) before they become a second informal ledger.

9. SOURCES (ONE-LINE VERDICTS)

Source	Verdict	One line
Karpathy, LLM Wiki gist (2026)	used	Compile-at-ingest wiki; index/log/lint; LLM as maintainer — pattern stolen, writer refused
OKF SPEC v0.2	used	Interchange format; trust-in-FM; Attested Computation; permissive conformance
Google Cloud blog 2026-06-12 (OKF intro)	used	Format not platform; Karpathy named; catalog-knowledge problem
Google Cloud blog 2026-07-24 (v0.2)	used	Five questions; signals not scores; verification ≠ attestation
thecodacus/understory (main, 2026-08)	used (pattern) / refused (product)	Worked OKF v0.1 + MCP wiki daemon
GoogleCloudPlatform/knowledge-catalog	skimmed	Product that ingests OKF; not the spec; not a PC store
understory-io/mcp	refused (wrong object)	Bookings API; same name
Folder 2 files 00–06	used	Grid this chat was scored against
About Memory, RAG, and Graphs	used	Karpathy row already: compiled Notes, never SoR
Revival v4 §4.1	used	Memory MCP / wiki daemon already refused
Huang ch. 2; Mercer ch. 12 (via folder 2 01 §20)	used	Instruction files vs opaque auto-memory — Understory is the transparent-files, agent-writer variant of the same problem

End of note. Sibling: [About Memory, RAG, and Graphs](#) (Karpathy row, MCP-as-transport). Folder 2 lives on the Nextcloud AI Reviews shelf; this file does not amend it.